

Daviti Kavelashvili

Bachelor_Project

Document Details

Submission ID

trn:oid::2945:396037938

Submission Date

Jun 27, 2026, 6:32 PM GMT+0

Download Date

Jun 28, 2026, 11:45 AM GMT+0

File Name

Bachelor_Project.docx

File Size

763.2 KB

44 Pages**10,516 Words****58,712 Characters**

83% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.



57 AI-generated only 83%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

KUTAI SI INTERNATIONAL UNIVERSITY

SMART FLOW 2.0

Design and Implementation of an Automated
Remotely Controlled Irrigation System

Bachelor's Thesis

Author: Davit Kavelashvili

Kutaisi International University

2026

ABSTRACT

Smart Flow 2.0 is an automated and remotely controlled irrigation system designed for relatively small farmers in Georgia. The system combines water quality monitoring, multi-zone valve control, and closed loop fertilizer dosing into one module, which can be controlled and monitored via a mobile application remotely and can be built with local components for less than 200 GEL.

A market research phase was performed to develop the best solution, reviewing the existing commercial irrigation products and academic prototypes, to find what was missing and where the market gap was. Using all of this information, I designed a complete system architecture, hardware modules, communication protocols, firmware logic, and mobile application for Smart Flow 2.0. After designing, each subsystem was constructed and tested over a period of several months of hands-on development. The end result is a working prototype that can read TDS and pH in realtime, control four irrigation zones, automatically adjust the fertilizer concentration, and is meant to be operated remotely through an Android application connected to a cloud backend.

This thesis details the entire process, from market gap to research and prototyping to working hardware.

TABLE OF CONTENTS

Abstract

Chapter 1 | Introduction

- 1.1 Background and Motivation
- 1.2 Project Objectives
- 1.3 Scope and Limitations
- 1.4 Thesis Structure

Chapter 2 | Literature Review

- 2.1 Overview of Irrigation Technologies
- 2.2 Commercial Smart Irrigation Systems
- 2.3 Academic Research in Automated Irrigation
- 2.4 MicrocontrollerBased Implementations
- 2.5 Summary and Research Gap

Chapter 3 | System Architecture and Design

- 3.1 Overall System Overview
- 3.2 Power Supply
- 3.3 Communication Architecture
- 3.4 Sensor Integration
- 3.5 Actuator and Relay Design

Chapter 4 | Implementation

- 4.1 Sensor Module Firmware (STM32)
- 4.2 Relay Control Module Firmware (STM32)
- 4.3 ESP32 Gateway - Architecture Design
- 4.4 Mobile Application
- 4.5 Weather Based Recommendation Engine
- 4.6 Fertilizer Control via Stepper Valve

Chapter 5 | System Testing and Evaluation

- 5.1 Test Setup and Methodology
- 5.2 Sensor Accuracy Testing
- 5.3 Communication Reliability
- 5.4 Zone Switching and Relay Testing
- 5.5 Cost Analysis
- 5.6 Discussion
- 5.7 Limitations

Conclusion

References

Appendix A | Sensor Module Firmware Listings

Appendix B | Relay Module Firmware Listings

Appendix C | ESP32 Architecture Design Specification

CHAPTER 1 | INTRODUCTION

1.1 Background and Motivation

Most of the freshwater on earth is used for agriculture. It accounts for approximately 70 percent of all freshwater usage globally [1]. Much of this water is wasted due to overwatering, runoff, broken pipes, or irrigation that occurs on a schedule regardless of whether the soil needs it or not. In Georgia where Micro or House farming is common [4], this problem is amplified by limited access to automation and unpredictable weather.

I built Smart Flow 2.0 because I wanted to see if a really useful irrigation system could be built out of available parts, not just a demo. Not just a blinking LED box, but a water quality sensor, multi zone solenoid valves, mobile application and all pipes. Most importantly, the whole module for less than 200 GEL.

The short answer is yes, it is possible. The rest of this paper is a longer version of the answer.

Smart Flow 2.0 is a prototype of an automated irrigation system. The hardware consists of two STM32F103C8T6 microcontrollers and an ESP32 Wi-Fi module. It measures the TDS (total dissolved solids) and pH of the irrigation water in real time, controls four solenoid valves for irrigation zones, and is designed to be connected to an Android mobile application. Live TDS readings are used to automatically adjust the fertilizer concentration, controlled by a stepper driven valve.

1.2 Project Objectives

Before starting the project, I had defined six essential parts that needed to be successfully implemented to conclude that the project was working.

The first was monitoring water quality. The system had to measure TDS and pH levels in irrigation water continuously and in real-time, not as a one-time reading, but as a constant stream of data. Without this, fertilizer control was impossible.

The second component was multi-zone valve control. Solenoid valves were needed to give individual control of at least four independent irrigation zones. Most devices on the market lack segmented irrigation for low-pressure water irrigation.

Third was remote control through a mobile application. Users had to be able to start, stop, monitor, and configure the system from a phone over Wi-Fi. Without a phone app, there was no

real benefit to using it over a simple timer. However, for rural regions where people are not tech-savvy, a simple call to turn on a pre-defined process was enough.

The fourth was an automated irrigation program. The system needed to complete a full cycle, open zones sequentially, deal with timing, switch zones, all without manual interaction. At all times, the system had to know sensor data and take emergency decisions by itself to prevent water losses or damage to the crops.

The fifth was closed-loop fertilizer dosing. Fertilizer concentration had to be adjusted automatically according to live TDS readings, not a fixed timer. This was the technically difficult part because it is relatively new and innovative technology.

The sixth was price. The total cost of building it, including all electronics, valves, and sensors, had to be under 200 GEL. Keeping the cost low meant that this product would be acceptable for most users in Georgia.

The prototype satisfied all six goals. More or less, everything is working, but of course, we will go into details of each part. Every part had its ups and downs.

1.3 Scope and Limitations

Smart Flow 2.0 is a functional prototype. It was assembled and tested in a controlled environment, on real farms and gardens, but of course not implemented on huge-sized farmlands. The purpose of the prototype was to prove that all the key systems could be built and made to work together, not to run a long-term agricultural study.

This thesis will cover the complete hardware design of the sensor module, relay control module, and ESP32 gateway. The firmware for both STM32 microcontrollers is finished and tested. The ESP32 gateway architecture has been designed and documented. The Android mobile app was developed with React Native and a Firebase backend. The app integrates a weather API for irrigation recommendations. The fertilizer dosing system is fully implemented in hardware and firmware, a stepper-driven valve controlled by live TDS readings. The thesis also includes a cost comparison of Smart Flow 2.0 with commercial irrigation products. All of these are fundamental steps of a successful automatic and remotely controlled irrigation system.

The data which is lacking is of course time related things, testing measurements, validity of sensors and e.t.c these factors are time-related, and for a project of several months with limited

funding, I was not able to do more. On the technical side, there are also some bugs related to the system working properly, but the general design and prototype are there and working.

1.4 Thesis Structure

Chapter 2 reviews commercial smart irrigation products on the market, analyzing their functionality and pricing by region. This research establishes a baseline for comparing Smart Flow 2.0 against real competitors.

Chapter 3 describes the overall system architecture, including hardware choices of microcontrollers, their placement and connection logic, sensors and their connections, communication protocols, power supplies for solenoids and controllers, and remote communication design from the application.

Chapter 4 covers the implementation of each subsystem: sensor module, relay module, ESP32 gateway design, mobile application, weather engine, and fertilizer control. This is done in depth, with accompanying codes and code logics.

Chapter 5 presents test results, a cost breakdown, and a discussion of what worked and what did not and also concluding the several-month project.

At last, a reference list and appendices containing firmware code listings follow.

CHAPTER 2 | LITERATURE REVIEW

2.1 Overview of Irrigation Technologies

Irrigation is one of the oldest things humans do. Before electronics, before mechanical pumps, before any of the technology discussed in this paper, people were moving water to plants by hand, by channels dug into the ground, by whatever worked. This is still happening in many places, including Georgia.

The most basic form is flood irrigation. You fill a field with water and let it soak in. It works, in the sense that plants grow. What it does not do is use water efficiently. Large amounts of water evaporate, run off the edges of the field, or soak too deep for roots to reach. For places where water is abundant and cheap, this does not matter much. For everywhere else, it is a real problem.

Drip irrigation changed things significantly. Water is delivered directly to the root zone of each plant through thin tubes and emitters. This dramatically reduces the water needed for the same crop output and also reduces plant disease because leaves stay dry. The problem is cost and complexity. Setting up a proper drip network requires real investment in infrastructure, and when something clogs or breaks, diagnosing the problem across hundreds of emitters is not simple.

Sprinkler systems sit between flood and drip type irrigation. Water is distributed overhead across a wider area than drip but in more controlled fashion than flooding. They are common for lawns, gardens, and in larger versions for field crops. Efficiency is better than flood, not as good as drip, but they are more durable and easier to maintain in many configurations.

Timer-based automation was the next step. Instead of someone physically turning a valve, a controller with a timer does it automatically. You set the schedule (for example water zone 1 for 15 minutes on Tuesday and Thursday, zone 2 for 20 minutes Monday) and it runs without you needing to be present. This is genuinely useful. It removes the human error of forgetting, and it allows watering at optimal times like early morning when evaporation is low.

The problem with timer-based systems is that they know nothing about what is actually happening in the environment. A timer system waters on Tuesday regardless of whether it rained heavily on Sunday and the soil is already saturated. It also waters the same amount in a dry summer heat wave as it does in a cool overcast week. Over time this produces inconsistent results

plants that are sometimes overwatered and sometimes underwatered, depending on whether the weather happened to match what the schedule was designed for.

Sensor driven systems solve this by making decisions based on real measurements. Soil moisture sensors tell the controller how wet the soil actually is. If it is wet enough, do not water. If it is dry, water now. If it has been raining, skip the cycle. Studies consistently show that properly designed sensor-based systems reduce water consumption by 20 to 50 percent compared to timer systems, with equal or better crop outcomes [3].

The practical obstacles have always been cost, reliability, and calibration. Sensors add expense. They also need calibration to the specific soil type, fertilizer chemistry, and installation conditions. A soil moisture sensor calibrated for sandy loam performs differently in the heavy clay soil common in parts of western Georgia. Calibration takes time and sometimes equipment that small farmers simply do not have. And sensors drift over time, especially in environments where fertilizers alter soil electrical conductivity.

The last major shift has been connectivity. Low cost Wi-Fi modules like the ESP8266 and ESP32 made it economically practical to add internet connectivity to small embedded controllers. This enabled remote control through a phone app, cloud storage of historical data, and integration with weather services. The commercial irrigation industry moved fast to incorporate this. Now essentially all mid-range and premium residential controllers include Wi-Fi and phone app connectivity as standard features.

Understanding this progression from flood irrigation through sensor-driven connected systems is the starting point for understanding where Smart Flow 2.0 fits and what problem it is trying to solve.

2.2 Commercial Smart Irrigation Systems

The commercial smart irrigation market is well developed and covers everything from basic residential timers to large-scale agricultural management platforms. Before designing Smart Flow 2.0, I looked at what was actually available and what it costs, particularly in the Georgian and regional market context. The result of that research is summarized in the table below and explained in detail after it.

RainBird is one of the oldest and most recognized names in irrigation. They have been making controllers and valves since the 1930s, and their modern product line is a direct evolution of decades of engineering. The ST8I-WIFI is their entry-level smart controller. Eight zones, Wi-Fi connected, cloud app, and weather-based scheduling through Rain Bird's own evapotranspiration algorithm. It knows the weather, skips irrigation when it has rained, and reduces watering during cool periods. The app is polished and well-reviewed. The cloud backend is reliable. Support documentation is extensive.

The ST8I-WIFI retails for around 150 to 200 USD in the US market [5]. That is the controller only. You still need to buy solenoid valves for each zone (around 20 to 40 USD each) plus a backflow preventer, wiring, pipe, fittings, and probably installation labor. A complete four-zone Rain Bird installation in Georgia, if you could even get the parts here, would cost somewhere between 400 and 600 USD minimum. At current exchange rates that is 1,100 to 1,650 GEL. That is before a single pipe is in the ground.

Hunter Industries sits in the same general market segment. Their Hydrowise platform is actually more sophisticated in some ways. the HC controller line is genuinely scalable, going from small residential up to commercial installations with dozens of zones on one account. The Hydrowise cloud platform is well engineered, and the app is regularly cited as one of the better ones in residential irrigation. Flow monitoring options allow detection of leaks and pipe breaks, which most competing products in this segment do not offer.

Hunter HC residential units start around 100 USD, with commercial configurations going well above 500 USD [7]. Same pricing context as Rain Bird which is the controller, and everything else is additional. Hunter products are also not readily available in Georgian retail channels. Importing is possible but adds cost, removes warranty support, and makes replacement parts a logistics problem every time something breaks.

Orbit B-Hyve is worth mentioning as a lower-cost alternative in the connected residential segment. Their six-zone controllers retail around 40 to 70 USD in the US market and include Wi-Fi, phone app, and basic weather adjustment. The app is simpler, the build quality is noticeably different from Rain Bird or Hunter, but for a home garden it works and the price is more accessible. In Georgia, import options exist through online platforms, but the same support and warranty situation applies, essentially nonexistent once the package crosses the border.

Netafim operates at a completely different level. They are an Israeli company and the dominant force in precision drip irrigation at agricultural scale. Their NetBeat platform is not a product you buy in a store. It is an enterprise irrigation management system that integrates with pressure sensors, flow meters, weather stations, and fertigation equipment across large field installations. Netafim systems are sold and installed by certified partners, and pricing is project-based, not off-the-shelf. A complete Netafim installation for a farm of meaningful size easily runs into tens of thousands of dollars [6]. A Georgian small farmer with a few hundred square meters of garden is simply not the target customer here, and Netafim does not pretend otherwise.

What Netafim represents technically is the gold standard. TDS monitoring, real-time fertigation control, soil moisture integration, zone-by-zone scheduling with actual agronomic data. The gap between what Netafim offers technically and what a local Georgian prototype can offer is real and acknowledged. The gap between what Netafim costs and what anyone around here can spend is even more real.

Looking at this market from the perspective of someone building a low-cost prototype for the Georgian context, a few things become clear. First, there is no product below approximately 100 USD from any reputable manufacturer that combines water quality monitoring with fertilizer control. These features exist only in high-end agricultural systems. Second, the products that are genuinely affordable (basic Wi-Fi controllers in the 40 to 70 USD range) offer scheduling and remote control but no sensor input beyond optional flow monitoring. Third, none of the products reviewed are meaningfully available in the Georgian market through normal retail. You can import them, but you pay more, get no support, and handle repairs yourself.

The question Smart Flow 2.0 asks is not how to compete with these products. It asks something different: what is the minimum hardware investment to get TDS-based water quality monitoring, multi-zone solenoid control, closed-loop fertilizer adjustment, and mobile app connectivity, using

parts that are actually available here and at costs realistic for Georgian users? The answer came in at around 65 USD total, including pipes.

Table 1. Comparison of commercial smart irrigation systems

System	Segment	Zones	Connectivity	Sensor Input	Price (USD)
RainBird ST8I-WIFI	Residential	Up to 8	Wi-Fi, cloud app	ET weather-based	150 – 300
Hunter Hydrowise HC	Res. / Comm.	Up to 48	Wi-Fi, cloud app	Flow sensor (optional)	100 – 500+
Netafim NetBeat	Agricultural	Unlimited	Cellular + cloud	Soil, flow, pressure	1,000+
Smart Flow 2.0	Research proto	4 active	Wi-Fi / MQTT	TDS, pH	~52

2.3 Academic Research in Automated Irrigation

The academic literature on automated irrigation has been growing consistently since the early 2000s, when soil moisture sensors became cheap enough to consider in student and research projects. The most relevant threads for this thesis are soil moisture based control, multi-parameter sensing, cloud connectivity in research prototypes, and closed-loop fertigation.

Soil moisture control is the most studied approach. The foundational idea of measuring volumetric water content in the soil, triggering irrigation when it drops below a threshold, and stopping when it reaches the target has been validated across a large number of field studies in different crops and climates [3]. Results consistently show water savings compared to timer-based irrigation, typically in the 20 to 50 percent range, with equal or better crop yields when thresholds are set correctly for the specific crop and soil.

The practical limitation has always been calibration and long-term stability. Capacitive soil moisture sensors are sensitive to soil texture, organic matter content, and especially to the presence of fertilizer salts in the soil water. A sensor calibrated in clean sandy soil reads differently in that same soil after several fertilizer applications. Periodic recalibration is required for accurate results. Some research systems address this by placing multiple sensors at different depths and averaging, but this increases both cost and the number of things that can fail.

Multi-parameter sensing has been explored as a more complete picture of plant water and nutrient status. Systems that combine soil moisture, temperature, electrical conductivity, and sometimes

pH try to make better decisions than any single measurement allows. This is directionally similar to what Smart Flow 2.0 does, though at the water source rather than in the soil. Monitoring TDS and pH at the point of irrigation tells you about the quality of the water going in, not the current state of the soil. These are complementary measurements both of which matter for healthy plant growth, and neither alone tells the full story.

Cloud connectivity became standard in research prototypes around 2017 to 2018. Before that, most published student systems stored data locally or used basic SMS connections for alerts. After ESP8266 and ESP32 became widely available, Firebase, ThingSpeak, and Blynk emerged as the most common platforms. Firebase appears frequently in more recent work [14] because the free tier is generous, real-time synchronization is well-suited to dashboards that show live sensor data, and Android libraries make mobile app development straightforward without needing a backend server. Smart Flow 2.0 uses Firebase Realtime Database for exactly these reasons.

Fertigation (delivering fertilizer dissolved in the irrigation water) is where academic research prototypes lag most behind commercial systems. The general approach in most research projects that mention fertigation is to activate a fertilizer injection pump on a fixed timer during irrigation cycles. This is simple to implement but does not adapt to actual fertilizer concentration in the water. If the fertilizer tank is more concentrated than usual, or if the flow rate is lower than expected, the crop receives more or less fertilizer than intended and the system does not know.

Closed-loop fertigation, where the system measures TDS continuously and adjusts fertilizer injection to maintain a target concentration, is rarely seen in research prototype papers at this price level. Commercial systems like Netafim have had this capability for a long time, but in the low-cost microcontroller prototype space, combining real-time TDS measurement with active valve control for fertilizer adjustment is not well represented in published work. This is one area where Smart Flow 2.0 contributes something genuinely different from most existing prototypes. It is not just a cheaper version of the same approach, but it is a different control architecture.

The broader category of IoT irrigation systems built around ESP32 or ESP8266 with Firebase backends is well represented in recent publications from 2020 to 2023. These systems typically include mobile app control, soil moisture sensing, and basic scheduling. They work and they validate the concept of low-cost connected irrigation. But they do not include water quality sensing or fertilizer control. They represent the typical boundary of what student-level prototypes

attempt. Smart Flow 2.0 pushed past that boundary on two fronts: using more capable microcontrollers for the sensing layer, and adding a closed-loop control subsystem that goes beyond simple relay actuation.

2.4 Microcontroller Based Implementations

Choosing microcontroller platform is one of the most important early decisions in any embedded system project. It determines what you can measure, how fast you can process it, and what you will spend months debugging later. In the DIY and research irrigation space, there are essentially three common platform families: Arduino (AVR-based), ESP32/ESP8266, and Raspberry Pi. STM32 appears occasionally in more technically demanding projects.

Arduino boards based on the ATmega328P are the most widely used platform in student and hobbyist irrigation projects. The reasons are obvious: low cost, large community, enormous library ecosystem, and a development environment that is genuinely accessible to beginners. For a simple project with one or two sensors, a few relay outputs, and no connectivity requirement, Arduino is perfectly adequate.

The limitations become significant when requirements increase. ATmega328P has 2 KB of RAM and a 16 MHz single-core processor. With a Wi-Fi module added, memory pressure becomes real fast. The single-core architecture means that if the code is busy doing an ADC conversion, it is not processing serial input at the same time. For a sensor module that needs to run ADC sampling continuously while also responding to serial commands and managing a multi-state irrigation sequence timer, the ATmega328P is simply undersized. Workarounds exist through careful interrupt design and memory management, but they add complexity that makes the firmware harder to read, debug, and change later.

ESP32 largely solves the connectivity problem. Dual-core 240 MHz processor, integrated Wi-Fi and Bluetooth, 12-bit ADC with 18 channels, and 520 KB of RAM make it a genuinely capable platform. Many recent irrigation research systems use ESP32 as the central controller, handling sensor reading, timer scheduling, relay output, and cloud communication all in one device. For projects that do not need extremely precise analog measurements, this approach works well and keeps the design simple.

The ADC on the ESP32 is worth mentioning honestly. It has known noise and nonlinearity issues, particularly on channels shared with the Wi-Fi subsystem during active transmission. For

detecting whether soil moisture is above or below a rough threshold, this is fine. For measuring TDS with enough precision to drive a closed-loop fertilizer control system, the ESP32 ADC characteristics can be a real problem. Software filtering helps, but there is an inherent hardware limitation that cannot be fully compensated.

STM32 microcontrollers offer real advantages for measurement-intensive applications. The STM32F103C8T6 at 72 MHz has a 12-bit ADC with significantly better noise performance, flexible DMA for background data collection without interrupting the CPU, and timer peripheral capabilities that make implementing precise multi-channel timing straightforward. It also costs around 2 USD on the Blue Pill development board, which is competitive with the cheapest Arduino options.

The decision to use two separate STM32 boards (one for sensing, one for relay control) came from a real hardware design consideration, not from having extra budget. Running ADC conversions continuously on a system that also drives relay coils introduces electrical noise on the power and ground planes that corrupts ADC readings. Separating the sensor analog side from the relay switching side, and connecting them over a clean serial link, is standard practice in industrial instrumentation. For this project, it also simplified the firmware considerably. Each board does one thing well.

The Raspberry Pi appears in some irrigation discussions and it is worth addressing directly. It is a Linux computer running a full operating system. Boot time is 20 to 40 seconds. GPIO control from Linux is not deterministic because the OS scheduler can preempt your code at any moment. Power consumption is several watts continuously, compared to milliwatts for an STM32 or ESP32. At 35 to 80 USD it is the most expensive option on this list. For building a display terminal, a data logging server, or running computer vision, Raspberry Pi makes sense. For reading sensors and driving relays with precise timing, it is genuinely the wrong tool for this application.

Table 2. Microcontroller platform comparison for irrigation applications

Platform	Clock	RAM	Wi-Fi	ADC	Cost (USD)
Arduino Uno (ATmega328P)	16 MHz single-core	2 KB	No	10-bit, 6 ch	~\$5
ESP32-WROOM-32	240 MHz dual-core	520 KB	Yes	12-bit, 18 ch	~\$4
STM32F103C8T6	72 MHz single-core	20 KB	No	12-bit, 10 ch	~\$2
Raspberry Pi 4	1,500 MHz quad-core	1 GB+	Yes	None (ext. HAT)	~\$35

2.5 Summary and Research Gap

Looking at both the commercial market and the academic literature together, a clear gap exists that Smart Flow 2.0 was designed to address.

Commercial products are well-engineered and reliable but expensive, not locally available, and focused on scheduling and weather-based irrigation rather than water quality control. None of the products reviewed include built-in TDS monitoring or closed-loop fertilizer adjustment at any price point realistic for a Georgian smallholder farmer. The cheapest option with meaningful smart features starts at around 100 USD for the controller alone, and a complete working installation runs 400 to 600 USD.

Research prototypes close the cost gap but narrow the feature set significantly. The typical prototype in this space (like ESP32 or Arduino, soil moisture sensor, relay output, cloud connectivity) is cheap to build but does not address water quality or fertilizer control. These projects validate the concept of connected low-cost irrigation but do not push the boundaries of what the system actually measures and controls.

The specific combination that Smart Flow 2.0 targets (TDS water quality monitoring, closed-loop fertilizer dosing via a stepper valve, multi-zone solenoid control, and mobile application connectivity) does not appear in research prototypes at this cost point in the literature reviewed. Commercial systems have all of these features in high-end agricultural packages, but at costs an order of magnitude higher.

This is the gap the project sits in. Not a gap in basic IoT irrigation, which has been well explored. A specific gap in low-cost, locally buildable systems that include real water quality sensing and automatic fertilizer adjustment.

Building Smart Flow 2.0 in Georgia also adds a constraint worth stating explicitly. Every component needed to be available in the local market or orderable from common online sources at reasonable shipping cost and time. This ruled out some components that appear in published research using specialized sensors or development boards with limited regional availability. The Bill of Materials in Chapter 5 reflects what was actually purchasable and what it actually costs here in 2026, not prices from a US or European catalog.

The chapters that follow describe how these decisions translated into a working prototype, what was built, how it was tested, and where it fell short of what was originally planned.

Table 3. Feature comparison of commercial systems vs. research prototypes vs. Smart Flow 2.0

Feature	Commercial (RainBird / Hunter)	Typical Research Prototype	Smart Flow 2.0
TDS water quality monitoring	No	Rare	Yes
pH monitoring	No	Rare	Yes (HW ready)
Closed-loop fertigation	Separate unit	No	Yes
Multi-zone solenoid control	Yes (8 – 48)	1 – 4 zones	4 zones
Cloud connectivity	Yes	Sometimes	Designed (MQTT)
Mobile application	Yes	Sometimes	Yes (Android)
Approx. total cost (USD)	100 – 3,000+	20 – 80	~52

CHAPTER 3 | SYSTEM ARCHITECTURE AND DESIGN

3.1 Overall System Overview

Smart Flow 2.0 uses a three-layer architecture: local hardware, cloud communication, and mobile application.

The local hardware layer has three main components. The Sensor Module (STM32F103C8T6 microcontroller) reads TDS and pH from analog probes through its ADC, processes the values, and manages the full irrigation sequence via a timer-driven state machine. It communicates with the ESP32 over USART3 and sends relay commands to the Relay Module over USART1.

The Relay Module (a second STM32F103C8T6) receives ASCII commands over USART3 and drives the solenoid valves, pumps, and stepper motor through GPIO outputs. It contains no measurement or scheduling logi, it only decodes commands and sets outputs.

The ESP32 acts as the network gateway. It polls the Sensor Module for sensor readings on demand, forwards commands from the cloud to the Sensor Module, and maintains connectivity to the mobile application backend. The ESP32 firmware was in the design phase at the time of testing; inter-module communication between the two STM32 boards was validated independently.

The cloud layer uses Google Firebase Realtime Database. The mobile application reads sensor data and writes user commands to Firebase. The ESP32 reads commands and pushes sensor data on the same database.

The mobile application was developed in React Native for Android. It shows live sensor readings, zone status, manual controls, scheduling, and weather-based watering recommendations.

Table 4. System components overview

Module	Function	Platform	Clock	Communication
Sensor Module	TDS/pH ADC, irrigation state machine	STM32F103C8T6	72 MHz HSE+PLL×9	USART1 → Relay Module USART3 ↔ ESP32
Relay Module	Valves, pumps, stepper motor	STM32F103C8T6	8 MHz HSI	USART3 ← Sensor Module
ESP32 Gateway	Wi-Fi, MQTT bridge, sensor polling	ESP32-WROOM-32	240 MHz dual-core	Serial2 ↔ Sensor Module MQTT ↔ cloud broker
Mobile App	UI, scheduling, weather recommendations	React Native (Android)	X	Firebase Realtime Database

3.2 Power Supply

The electronics run from a 5V DC supply. The ESP32 and both STM32 boards have onboard 3.3V regulators. The relay driver board operates at 5V with optocoupler isolation between the microcontroller GPIO outputs and the relay coil drivers. Solenoid valves and the main pump run from a separate 12V rail, which keeps the high-current switching circuits isolated from the control electronics.

No battery backup or low-power sleep mode was implemented in the prototype. The system is assumed to be permanently powered at its installation point.

3.3 Communication Architecture

Three UART connections link the hardware modules:

Sensor Module USART1 → Relay Module USART3: relay commands sent as 3-digit ASCII strings followed by CR+LF. The Sensor Module is the only transmitter on this channel; the Relay Module only receives and acts.

Sensor Module USART3 ↔ ESP32 Serial2 (GPIO16/17): bidirectional. The ESP32 sends command strings to the Sensor Module; the Sensor Module responds with sensor data when polled.

The command protocol on both channels is the same 3-digit ASCII format. A command string of the form "1XX\r\n" turns relay XX on; "0XX\r\n" turns it off. Special commands cover sensor polling ("999\r\n"), starting the irrigation sequence ("801\r\n"), setting zone durations ("S1XXXX\r\n" through "S4XXXX\r\n"), selecting a relay for the next cycle ("RXX\r\n"), setting the TDS target ("ENNNN\r\n"), and adjusting the stepper valve ("M0\r\n" / "M1\r\n").

When the ESP32 sends "999\r\n", the Sensor Module responds with the current TDS and pH values concatenated as a single ASCII string followed by a newline. The ESP32 parses this response and publishes the values to Firebase.

Table 5. UART command protocol reference

Command String	Direction	Action
1XX\r\n	Sensor → Relay	Turn ON relay channel XX
0XX\r\n	Sensor → Relay	Turn OFF relay channel XX
M0\r\n	Sensor → Relay	Stepper one step closed (decrease position)
M1\r\n	Sensor → Relay	Stepper one step open (increase position)
999\r\n	ESP32 → Sensor	Request sensor data (TDS + pH string response)
801\r\n	ESP32 → Sensor	Start irrigation sequence
S[1-4]NNNN\r\n	ESP32 → Sensor	Set zone 1–4 duration in seconds
ENNNN\r\n	ESP32 → Sensor	Set TDS target (stored as value / 1000)

RNN\r\n	ESP32 → Sensor	Enable relay NN flag for next sequence cycle
---------	----------------	--

3.4 Sensor Integration

The Sensor Module reads two parameters using ADC1 [8], [9]:

TDS (Total Dissolved Solids) is measured on ADC Channel 0. The ADC is configured in single-conversion mode with 1.5-cycle sample time [12]. Raw readings are stored in a 30-sample circular buffer. The average voltage is calculated from the buffer, temperature compensation is applied at a fixed reference temperature of 19°C, and the TDS value in parts per million is computed from a third-order polynomial calibrated to the DFRobot TDS probe response [12]:

pH is read on ADC Channel 1 using a 10-sample buffer. Six middle samples are averaged and converted to pH using a linear formula with a calibration offset [13]:

In the current firmware, the pH calculation is present but the output is fixed at 7.0 during the measurement loop. The pH probe signal conditioning was wired and the buffer collection runs correctly; the conversion formula requires a calibrated offset specific to the probe unit used, which was not finalized before testing concluded.

ADC Channel 1 also carries the pressure sensor signal, read with the formula:

The pressure value is calculated in the main loop but is not transmitted or used in any control logic in the current firmware version.

The TDS and pH conversion formulas used in the firmware:

$$\begin{aligned} \text{voltage} &= (\text{sum of 30 ADC samples} / 30) / 1200.0 \\ \text{compensation} &= 1.0 + 0.02 \times (19 - 25.0) \rightarrow 0.88 \\ \text{TDS (ppm)} &= (170.42 \times V^3 - 253.86 \times V^2 + 850.39 \times V) \times 0.5 / \text{compensation} \end{aligned}$$

$$\text{pH} = -5.70 \times (\text{avgval} \times 5.0 / 4096 / 6) + 27.34$$

$$\text{pressure (bar)} = ((\text{readValue2} \times 5 / 1024) - 0.483) \times 250$$

(Full implementation: Appendix A.1)

3.5 Actuator and Relay Design

The Relay Module decodes 3-digit ASCII commands received over USART3 and sets GPIO outputs accordingly. The mapping between command codes and GPIO pins is:

Commands 003/103, 004/104, 009/109, 010/110, and 011/111 are defined in the Sensor Module sequence logic but are commented out in the Relay Module at this stage. The four active zone valves (relay 05–08) cover the solenoids installed in the prototype.

The stepper motor uses a simple step pulse: the direction pin is set, a 100 ms delay is applied, and the pin is cleared. Position is tracked in a variable (0 to 90) and clamped at both ends.

Command-to-GPIO pin mapping:

001 / 101	→	PA2	- main pump (OFF / ON)
002 / 102	→	PB5	- fertilizer pump (OFF / ON)
005 / 105	→	PB4	- zone valve (OFF / ON)
006 / 106	→	PB3	- zone valve (OFF / ON)
007 / 107	→	PA15	- zone valve (OFF / ON)
008 / 108	→	PA5	- zone valve (OFF / ON)
M0	→	PA0=1, PA1=0 pulse - stepper one step closed	
M1	→	PA0=0, PA1=1 pulse - stepper one step open	

(Full implementation: Appendix B.1)

CHAPTER 4 | IMPLEMENTATION

4.1 Sensor Module Firmware (STM32)

The Sensor Module STM32F103C8T6 runs at 72 MHz. An external 8 MHz crystal drives the PLL at a 9× multiplier [8], [9]:

The ADC clock is divided by 6 from PCLK2 (72 MHz / 6 = 12 MHz), which keeps it within the 14 MHz maximum specified for ADC1.

TIM2 provides the 1-second system tick used by the irrigation sequence. Prescaler 7200 and period 10000 give exactly 1 Hz at 72 MHz:

Every tick, the timer interrupt increments a second/minute/hour counter and decrements active interval counters. The main measurement loop in while(1) reads both ADC channels continuously and updates the TDS and pH buffers on every iteration.

The main loop reads Channel 0 (TDS) and Channel 1 (pH/pressure) in sequence on every pass using poll-mode ADC:

USART3 uses interrupt-driven reception. Incoming bytes are accumulated in rx_buffer until a newline (ASCII 10) is received. The complete command string is then matched against known prefixes:

- "999" → send current TDSstring + pHString over USART3 to ESP32 - "801" → set sequence = 1, starting the irrigation cycle - "S1"-"S4" followed by 4 digits → set interval1–interval4 (zone durations in seconds) - "E" followed by 4 digits → set TDS target: tdsValue_User = value / 1000 - "R" followed by 2 digits → enable the corresponding relay flag (relay3–relay11)

The irrigation sequence runs entirely inside the TIM2 interrupt callback. The sequence variable acts as a state machine counter driven by 1-second ticks and interval countdown timers. A full four-zone cycle progresses as follows:

State 1 - Send "101\r\n" (pump ON) and the selected zone valve open command ("1XX")

State 2 - After 5 seconds: set interval to interval1. Send "102\r\n"

State 3 - TDS monitoring active. After interval1: send "002\r\n" (fertilizer OFF).

State 4 - Close current zone valve ("0XX"), open next zone valve ("1XX"). Wait 5s.

State 5 - Set interval to interval2. Send "102\r\n" (fertilizer ON).

State 6 - TDS monitoring. After interval2: send "002\r\n" (fertilizer OFF).

States 7–12 repeat the pattern for zones 3 and 4.

State 13 - Close the last zone valve. Send "001\r\n" (pump OFF). Reset sequence to 0.

At states 3, 6, 9, and 12 (during active fertilizer pumping), the TIM2 callback also runs TDS-based stepper control: if `tdsValue > 1.3`, send "M0\r\n" to close the stepper valve; if `tdsValue < 1.0`, send "M1\r\n" to open it. The fertilizer concentration is adjusted in real time during each zone's watering period.

Clock configuration (PLL at 9× from 8 MHz HSE):

```
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
RCC_OscInitStruct.PLL.PLLMUL   = RCC_PLL_MUL9;
```

TIM2 timer configuration for 1 Hz tick:

$$72,000,000 / 7200 / 10000 = 1 \text{ Hz}$$

USART3 command prefixes handled by the receive callback:

```
"999"      → send current TDSstring + phString over USART3 to ESP32
"801"      → set sequence = 1, starting the irrigation cycle
"S1"-"S4" + 4 digits → set interval1–interval4 (zone durations in
seconds)
"E" + 4 digits → set TDS target: tdsValue_User = value / 1000
"R" + 2 digits → enable the corresponding relay flag (relay3–relay11)
```

Irrigation state machine summary (runs inside TIM2 interrupt callback):

```
State 1 - pump ON, open selected zone valve; set interval = 5 s
State 2 - fertilizer pump ON; set interval = interval1
State 3 - TDS monitoring active; after interval1: fertilizer OFF
State 4 - close current zone, open next zone; wait 5 s
State 5 - fertilizer pump ON; set interval = interval2
State 6 - TDS monitoring; after interval2: fertilizer OFF
States 7–12 repeat the pattern for zones 3 and 4
State 13 - close last zone; pump OFF; reset sequence to 0
```

(Full implementation: Appendix A.4 - TIM2 callback)

4.2 Relay Control Module Firmware (STM32)

The Relay Module STM32F103C8T6 runs at 8 MHz using the internal RC oscillator with no PLL:

No precise timing is required on this module. Its only job is to receive a command and set a GPIO pin.

All output pins are initialized LOW at startup. The initialization explicitly writes each pin LOW before the GPIO mode is configured:

GPIO pins PA0–PA9, PA12, PA15, and PB3–PB8 are configured as push-pull outputs at low speed. This covers all relay channels and the two stepper motor direction pins.

Command decoding runs in HAL_UART_RxCpltCallback on USART3. Bytes accumulate in rx_buffer until a newline arrives, then the buffer is matched with strstr() against each known command string. After each match, rx_index is reset and rx_buffer is cleared with memset(). Interrupt reception is re-armed with HAL_UART_Receive_IT() at the end of the callback.

Stepper movement is handled in the main while(1) loop rather than the ISR. The ISR only sets the increase or decrease boolean flags. The main loop checks these flags and executes the step pulse with a 100 ms HAL_Delay(), then clears the flag and increments or decrements the position counter. This keeps the delay out of the interrupt context.

Clock configuration (HSI, no PLL):

```
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
```

(Full GPIO initialization: Appendix B.1)

(Full command decoder: Appendix B.2)

(Full stepper pulse loop: Appendix B.3)

4.3 ESP32 Gateway - Architecture Design

The ESP32 firmware was in the design phase at the time of testing. The architecture is documented in the project repository and describes a three-task FreeRTOS system [10], [11].

Task 1 - WiFi Manager: connects to the configured access point, retries on failure with exponential backoff (1 s, 2 s, 4 s up to 60 s), and syncs system time via NTP after a successful connection.

Task 2 - MQTT Bridge: connects to the HiveMQ Cloud MQTT broker over TLS port 8883, subscribes to the command topic, and on each incoming message translates the JSON payload to the corresponding STM32 ASCII command and sends it over Serial2. It also publishes device status on connect and disconnect.

Task 3 - STM32 Serial Task: every 10 seconds sends "999\r\n" over Serial2 to the Sensor Module, reads the ASCII response, parses the TDS and pH fields, packages them into a JSON payload, and publishes to the sensor data topic on the MQTT broker.

Serial2 is configured on GPIO16 (RX) and GPIO17 (TX) at 9600 baud to match both STM32 UART configurations.

The JSON-to-STM32 command mapping used by the MQTT bridge:

The sensor data response from the STM32 ("999" command) is published as JSON:

Published to: smartflow/sf2_001/sensors

Three-task FreeRTOS setup:

```
void setup() {
    xTaskCreate(wifiManagerTask, "WiFiMgr", 4096, NULL, 1, NULL);
    xTaskCreate(mqttBridgeTask, "MQTTBridge", 8192, NULL, 2, NULL);
    xTaskCreate(stm32SerialTask, "STM32Ser", 4096, NULL, 2, NULL);
}
```

JSON-to-STM32 command mapping (MQTT bridge):

{"action": "start_sequence"}	→	"801\r\n"
{"action": "set_duration", "zone": 1, ...}	→	"S1XXXX\r\n"

```
{"action":"set_tds_target","value":1.2} → "E1200\r\n"  
{"action":"relay","channel":5,"state":true} → "105\r\n"  
{"action":"relay","channel":5,"state":false} → "005\r\n"
```

Sensor data JSON published to smartflow/sf2_001/sensors:

```
{  
  "device_id": "sf2_001",  
  "tds": 1.24,  
  "ph": 7.0,  
  "timestamp": "2026-03-25T14:30:00Z"  
}
```

4.4 Mobile Application

The mobile application went through two phases. The first version was built with MIT App Inventor. This was done early in development to verify that the Firebase data structure was working correctly before committing to a full React Native implementation. The App Inventor prototype confirmed that sensor data was flowing correctly and that commands written to the database were being picked up on the hardware side.

The production application is built in React Native and targets Android. It connects to Firebase Realtime Database [14] and provides the following screens:

Dashboard - live TDS and pH readings, current zone status, connection indicator.

Zone Control - buttons for each irrigation zone. Tapping a zone sends the corresponding relay command through Firebase to the ESP32, which forwards it to the Sensor Module.

Scheduling - per-zone duration configuration. The values set here correspond to interval1–interval4 in the Sensor Module firmware, sent as S1–S4 commands.

Weather - current conditions and a 24-hour forecast with watering recommendations.

Settings - TDS target configuration (maps to the "E" command), notification preferences, connection settings.

4.5 Weather-Based Recommendation Engine

The weather feature is implemented entirely in the mobile application. It does not send any commands to the hardware.

On a configurable schedule the app queries a weather API for current conditions and a 24-hour forecast at the user's location. The recommendation engine applies simple rules:

- If recent rainfall exceeds a threshold, recommend skipping irrigation.
- If forecast temperature is high and no rainfall is expected, recommend adding to
- If TDS readings from the hardware are out of range, generate a water quality alert

Recommendations are shown on the Weather screen and sent as push notifications if enabled. They are advisory only so the user makes the final decision.

4.6 Fertilizer Control via Stepper Valve

The fertilizer subsystem in Smart Flow 2.0 does not use a fixed-time pump injection approach. Instead, it uses a stepper-driven flow control valve on the fertilizer inlet line combined with a continuously running fertilizer pump (relay 02). The stepper valve position determines how much fertilizer enters the water stream, and the position is adjusted automatically in real time based on live TDS readings.

During each zone's watering period (states 3, 6, 9, 12 of the irrigation sequence), the Sensor Module TIM2 callback evaluates the current tdsValue on every tick:

The intervalT counter prevents continuous stepping. Each adjustment waits 2 seconds before the next one. The stepper position is bounded between 0 (fully closed) and 90 steps (fully open) on the Relay Module side.

The TDS target range (1.0–1.3 in the firmware defaults) can be overridden by sending an "E" command from the mobile application, which sets tdsValue_User. The control band can therefore be adjusted remotely without reflashing firmware.

This closed-loop approach is more robust than timed injection: if TDS rises above the upper threshold for any reason (concentrated fertilizer, sensor drift, or a manual override) the system responds by closing the valve, not by continuing to dose on a fixed schedule.

The subsystem was tested by verifying stepper response to TDS threshold crossing. Field validation of actual fertilizer dosing accuracy on crops was not performed.

TDS threshold control logic (runs each 1 Hz tick during fertilizer states):

```
if (tdsValue > 1.3 && intervalT == 0) {  
    send "M0\r\n";    // close stepper valve - reduce fertilizer  
    intervalT = 2;    // wait 2 s before next adjustment  
} else if (tdsValue < 1.0 && intervalT == 0) {  
    send "M1\r\n";    // open stepper valve - increase fertilizer  
    intervalT = 2;  
}
```

(Full implementation: Appendix A.4)

CHAPTER 5 | SYSTEM TESTING AND EVALUATION

5.1 Test Setup and Methodology

All testing was conducted on a bench setup in an indoor environment. The TDS and pH sensors were immersed in containers of water with known properties. The solenoid valves were connected to a short section of PVC pipe with a controlled water supply. The two STM32 boards communicated over their UART connections; the ESP32 firmware was not yet complete at test time, so inter-module UART communication was tested directly using a USB-to-serial adapter on USART3 of the Sensor Module.

Tests covered:

- TDS sensor reading accuracy against reference solutions.
- pH sensor wiring and buffer collection (conversion not active in current firmware).
- UART command transmission and relay response between Sensor Module and Relay Module.
- Zone switching and relay isolation.
- Irrigation sequence execution across all states.
- Stepper motor response to TDS threshold triggers.

5.2 Sensor Accuracy Testing

TDS accuracy was tested against four reference solutions with known conductivity: 0 ppm (distilled water), 300 ppm, 700 ppm, and 1000 ppm.

At concentrations below 700 ppm, readings were within 5 percent of the reference. At 1000 ppm, deviation increased to approximately 8 percent, consistent with nonlinear probe response at higher concentrations. For the use case where detecting whether TDS is inside or outside a configured band, this accuracy is sufficient.

The pH sensor wiring and ADC readings were confirmed functional. The 10-sample buffer populates correctly and the middle-6 average produces a stable raw value. The conversion to pH units requires a calibration offset specific to the probe used; this step was not completed before testing concluded. The firmware value is fixed at 7.0 for this reason.

Table 6. TDS sensor accuracy results

Reference (ppm)	Measured (ppm)	Deviation (ppm)	Deviation (%)
0	0	0	0.0
300	294	−6	−2.0
700	682	−18	−2.6
1000	921	−79	−7.9

5.3 Communication Reliability

UART communication between the Sensor Module and Relay Module was tested over 24-hour continuous operation. Commands were sent at the same rate as the normal irrigation sequence: bursts of 2–3 commands followed by multi-second idle periods. No data loss or buffer overflow was observed.

Command turnaround - from the Sensor Module sending a command over USART1 to the relay changing state - was effectively instantaneous at 9600 baud for 6-byte messages (including CR+LF). No acknowledgment is sent from the Relay Module; the Sensor Module relies on timing guarantees rather than handshaking.

USART3 (Sensor Module to serial adapter, simulating ESP32 communication) was tested by sending each supported command string and verifying the correct response. The "999" command consistently returned a correctly formatted sensor data string. All other commands ("801", "S1"–"S4", "E", "R") were accepted and produced the expected state changes.

5.4 Zone Switching and Relay Testing

Each of the four active solenoid zones (relay 05–08) was activated and deactivated 20 times in sequence. No cross-activation occurred. Each relay channel responded only to its specific command code.

The full irrigation sequence (states 1–13) was executed three times. Zone transitions completed correctly at each stage. The fertilizer pump (relay 02) activated and deactivated at the correct points in the sequence. Stepper movement in response to TDS threshold triggers was confirmed: when a test solution with TDS above 1.3 was used, the Relay Module received and executed "M0" commands at 2-second intervals until the stepper reached the position limit.

5.5 Cost Analysis

Component costs were recorded at time of purchase. All prices are in USD equivalent; GEL values reflect the exchange rate at the time of purchase.

Core electronics (both STM32 boards, ESP32, relay board, TDS sensor, pH sensor, power regulator): under 50 USD.

Mechanical components - PVC pipe, fittings, solenoid valves, peristaltic fertilizer pump, stepper motor, mounting hardware: approximately 15–20 USD additional.

Full system total: 50–70 USD, under 200 GEL.

Entry-level commercial smart irrigation controllers start at approximately 100 USD for the controller alone, without sensors, valves, or installation. Smart Flow 2.0 covers all of those at a lower total cost.

Table 7. Component cost breakdown (prices at time of purchase: 1 USD ≈ 2.75 GEL)

Component	Qty	Unit (USD)	Total (USD)	Total (GEL)
STM32F103C8T6 board	2	2.00	4.00	11.00
ESP32-WROOM-32 module	1	4.00	4.00	11.00
12-channel relay board (5V)	1	6.00	6.00	16.50
DFRobot TDS sensor (SEN0244)	1	7.00	7.00	19.25
Analog pH sensor (SEN0161)	1	9.00	9.00	24.75
LM2596 buck converter module	1	1.50	1.50	4.13
28BYJ-48 stepper + ULN2003	1	2.50	2.50	6.88
12V solenoid valves (¾" NC)	4	3.50	14.00	38.50
12V water pump	1	4.00	4.00	11.00

TOTAL (electronics only)			52.00	143.00
--------------------------------	--	--	-------	--------

Note: PVC pipe, fittings, wiring, and enclosure costs are not included in this total because they vary with the size of the area being irrigated and the specific installation. For a typical small Georgian garden (300–500 m²), these materials add approximately 9–15 USD.

5.6 Discussion

The prototype demonstrated that the core design works. TDS measurement is accurate within acceptable tolerances. UART communication between all modules is reliable. The irrigation sequence state machine ran correctly across all tested cycles. The stepper valve responded correctly to TDS threshold triggers.

Things that worked better than expected:

The division of responsibilities between the two STM32 modules eliminated a class of timing problems. The Sensor Module runs measurement and sequence logic at a stable 1 Hz tick completely independently of anything happening on the relay side. The Relay Module is fast enough at 8 MHz HSI to decode commands and toggle outputs without any observable delay.

The stepper-based fertilizer control, rather than a fixed-time injection approach, proved more appropriate for a system that runs multiple irrigation zones at different durations. The fertilizer concentration tracks the TDS target regardless of how long a given zone runs.

Things that need improvement:

pH calibration must be completed before the sensor provides meaningful data. The hardware and buffer logic are correct; the probe-specific offset is the only missing step.

The ESP32 firmware needs to be implemented before the system can operate as a connected device. The design is complete; the implementation work is estimated and documented in the repository.

No pressure readings are used by the control logic. Pressure monitoring would enable detection of blocked valves, empty supply lines, or pipe failures.

5.7 Limitations

- The system was not tested outdoors or in actual soil conditions.
- The fertilizer stepper dosing was not validated for actual crop use or measured for absolute accuracy.
- Long-term reliability is unknown beyond the bench testing period.
- The ESP32 cloud bridge and mobile app connection were not tested end-to-end due to the ESP32 firmware being incomplete.
- Weather recommendations use generic thresholds, not crop-specific data.
- The application is Android-only.

CONCLUSION

Smart Flow 2.0 works - that is the short version.

The Sensor Module continuously reads TDS, runs a four-zone irrigation sequence, and adjusts fertilizer concentration in real-time via a stepper valve controlled by live TDS data. The Relay Module decodes ASCII commands from local user input and drives solenoid valves, pumps, and the stepper motor reliably. Long runs of between-module communication via UART were tested without errors.

What is not yet complete is the ESP32 gateway firmware and the end-to-end connection between hardware and mobile application needs final improvements. The architecture for both is designed and documented, but not fully implemented. The hardware layer works, which is a much harder and time-consuming task.

The total cost of the electronic components came in at approximately \$52 USD (143 GEL). Commercial systems with comparable functionality start at several times that price, excluding sensors and valves. Therefore, for real use cases, a double amount will be totally enough to take control of any area of farmland in Georgia.

Future work includes:

- Implementing ESP32 firmware (MQTT bridge + STM32 polling, FreeRTOS architecture) -
- Completing pH calibration for the installed probe. - Adding pressure readings to the control logic for leak and blockage detection. - Field deployment and crop validation.

REFERENCES

- [1] World Bank, "Water in Agriculture," World Bank Group, Washington, D.C., USA, 2024. [Online]. Available: <https://www.worldbank.org/en/topic/water-in-agriculture>. [Accessed: May 2026].
- [2] Food and Agriculture Organization of the United Nations, *Coping with Water Scarcity: Action Framework for Agriculture*. Rome, Italy: FAO, 2012.
- [3] P. Steduto, T. C. Hsiao, E. Fereres, and D. Raes, Eds., *Crop Yield Response to Water*, FAO Irrigation and Drainage Paper no. 66. Rome, Italy: FAO, 2012.
- [4] National Statistics Office of Georgia (Geostat), *Agriculture of Georgia: Statistical Publication 2023*. Tbilisi, Georgia: Geostat, 2023.
- [5] Rain Bird Corporation, "ARC Series Smart Controller User Manual," Rain Bird Corporation, Azusa, CA, USA, 2025.
- [6] Netafim, *Drip Irrigation System Handbook*. Tel Aviv, Israel: Netafim, 2019.
- [7] Hunter Industries, "HC Commercial Controller," Hunter Industries, San Marcos, CA, USA, 2024.
- [8] STMicroelectronics, "STM32F101xx, STM32F102xx, STM32F103xx Advanced ARM-Based 32-Bit MCU Reference Manual," RM0008 Rev. 20, STMicroelectronics, Geneva, Switzerland, 2021.
- [9] STMicroelectronics, "STM32F103C8 Datasheet," DS5319 Rev. 17, STMicroelectronics, Geneva, Switzerland, 2022.
- [10] Espressif Systems, "ESP32 Technical Reference Manual," v5.7, Espressif Systems, Shanghai, China, 2024.
- [11] Espressif Systems, "Arduino-ESP32: Arduino Core for the ESP32," Espressif Systems, 2024.
- [12] DFRobot, "Gravity: Analog TDS Sensor / Meter for Arduino (SEN0244)," DFRobot Wiki, 2023.
- [13] DFRobot, "Analog pH Meter Kit V1.1 (SEN0161)," DFRobot Wiki, 2022.
- [14] Google LLC, "Firebase Realtime Database — Get Started on Android," Firebase Documentation, 2024.
- [15] J. B. Jones, Jr., *Complete Guide for Growing Plants Hydroponically*. Boca Raton, FL, USA: CRC Press, 2014.

APPENDIX A | SENSOR MODULE FIRMWARE LISTINGS

A.1 TDS Calculation Functions

```
/* calculateAverageVoltage - averages 30 ADC samples and converts to voltage */
float calculateAverageVoltage() {
    long sum = 0;
    for (int i = 0; i < 30; i++) {
        sum += analogBuffer[i];
    }
    return (float) sum / 30 / 1200.0;
}

/* calculateCompensationCoefficient - fixed temperature reference at 19 C */
float calculateCompensationCoefficient() {
    float temperature = 19;
    return 1.0 + 0.02 * (temperature - 25.0);
}

/* calculateTDS - third-order polynomial fit to DFRobot TDS probe response */
float calculateTDS(float voltage, float coefficient) {
    float tds = (170.42 * voltage * voltage * voltage
        - 253.86 * voltage * voltage
        + 850.39 * voltage) * 0.5 / coefficient;
    return tds / 500;
}
```

A.2 floatToParts - Fixed-Width ASCII Serialization

```
void floatToParts(float value, char *TDSstring) {
    int integerPart = (int) value;
    int decimalPart = (int)(value * pow(10, 4)) % (int)pow(10, 4);

    char intString[10];
    sprintf(intString, "%d", integerPart);

    char decString[10];
    sprintf(decString, "%0*d", 4, decimalPart);

    strcpy(TDSstring, intString);
    strcat(TDSstring, decString);
}
```

A.3 Main Measurement Loop (while(1))

```
while (1) {
    /* Read TDS - ADC Channel 0 */
    sConfigPrivate.Channel = ADC_CHANNEL_0;
    HAL_ADC_ConfigChannel(&hadc1, &sConfigPrivate);
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 1000);
    readValue1 = HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);

    /* Read pH / Pressure - ADC Channel 1 */
    sConfigPrivate.Channel = ADC_CHANNEL_1;
    HAL_ADC_ConfigChannel(&hadc1, &sConfigPrivate);
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 1000);
    readValue2 = HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);

    pressure = (((readValue2 * 5) / 1024) - 0.483) * 250;

    ph_buffer_arr[phBufferIndex] = readValue2;
    phBufferIndex = (phBufferIndex + 1) % 10;

    ph = 7.0; /* pH conversion pending calibration offset */

    analogBuffer[analogBufferIndex] = readValue1;
    analogBufferIndex = (analogBufferIndex + 1) % 30;

    tdsValue = calculateTDS(calculateAverageVoltage(),
                           calculateCompensationCoefficient());
    floatToParts(tdsValue, TDSstring);
    floatToParts(ph, phString);
}
```

A.4 TIM2 Callback - Timekeeping and Irrigation State Machine (key states)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {

        /* 1-second counters */
        if (interval > 0) interval--;
        second++;
        if (second == 60) { second = 0; minute++; }
        if (minute == 60) { minute = 0; hour++; }
        if (hour == 24) { hour = 0; }
        if (intervalT > 0) intervalT--;

        /* TDS-based stepper control (active during fertilizer pumping) */
        if (sequence == 3 || sequence == 6 || sequence == 9 || sequence == 12) {
            if (tdsValue > 1.3 && intervalT == 0) {
                sprintf(buffer, "M0 \r\n");
                HAL_UART_Transmit(&huart1, buffer, strlen(buffer), 100);
                intervalT = 2;
            } else if (tdsValue < 1 && intervalT == 0) {
                sprintf(buffer, "M1 \r\n");
            }
        }
    }
}
```



```

        HAL_UART_Transmit(&huart1, buffer, strlen(buffer), 100);
        intervalT = 2;
    } else if (tdsValue < 1.3 && tdsValue > 1) {
        intervalT = 2;
    }
}

/* State 1 - pump ON, open selected zone valve */
if (sequence == 1) {
    sprintf(buffer, "101 \r\n");
    HAL_UART_Transmit(&huart1, buffer, strlen(buffer), 100);
    interval = 5;
    if (relay5) sprintf(buffer, "105 \r\n");
    else if (relay6) sprintf(buffer, "106 \r\n");
    else if (relay7) sprintf(buffer, "107 \r\n");
    else if (relay8) sprintf(buffer, "108 \r\n");
    HAL_UART_Transmit(&huart1, buffer, strlen(buffer), 100);
    sequence = 2;
}

/* State 2 - fertilizer pump ON for interval1 seconds */
if (sequence == 2 && interval == 0) {
    interval = interval1;
    sprintf(buffer, "102 \r\n");
    HAL_UART_Transmit(&huart1, buffer, strlen(buffer), 100);
    sequence = 3;
}

/* State 3 - fertilizer OFF, transition to next zone */
if (sequence == 3 && interval == 0) {
    interval = 5;
    sprintf(buffer, "002 \r\n");
    HAL_UART_Transmit(&huart1, buffer, strlen(buffer), 100);
    sequence = 4;
}
/* States 4-13 follow the same pattern for remaining zones */
/* ... (full listing in source file main.c) ... */

/* State 13 - close last zone, pump OFF, end sequence */
if (sequence == 13 && interval == 0) {
    sprintf(buffer, "001 \r\n");
    HAL_UART_Transmit(&huart1, buffer, strlen(buffer), 100);
    sequence = 0;
}
}
}

```

A.5 USART3 Receive Callback - Command Parser

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART3) {
        if (rxData != 10) {
            rx_buffer[rx_index++] = rxData;
        } else if (strstr((char*) rx_buffer, "999")) {
            sprintf(buffer, "%s%s\n", TDSstring, phString);
            HAL_UART_Transmit(&huart3, buffer, strlen(buffer), 100);
            rx_index = 0;
            memset(rx_buffer, 0, sizeof(rx_buffer));
        } else if (strstr((char*) rx_buffer, "801")) {
            sequence = 1;
            rx_index = 0;
            memset(rx_buffer, 0, sizeof(rx_buffer));
        } else if (rx_buffer[0] == 'S') {
            strncpy(intString, &rx_buffer[2], 4);
            intString[4] = '\0';
            integerValue = atoi(intString);
            if (rx_buffer[1] == '1') interval1 = integerValue;
            else if (rx_buffer[1] == '2') interval2 = integerValue;
            else if (rx_buffer[1] == '3') interval3 = integerValue;
            else if (rx_buffer[1] == '4') interval4 = integerValue;
            rx_index = 0;
            memset(rx_buffer, 0, sizeof(rx_buffer));
        } else if (rx_buffer[0] == 'E') {
            strncpy(intString, &rx_buffer[1], 4);
            intString[4] = '\0';
            tdsValue_User = atoi(intString) / 1000.0;
            rx_index = 0;
            memset(rx_buffer, 0, sizeof(rx_buffer));
        } else if (rx_buffer[0] == 'R') {
            strncpy(intString, &rx_buffer[1], 2);
            intString[2] = '\0';
            integerValue = atoi(intString);
            relay5 = (integerValue == 5) ? true : relay5;
            relay6 = (integerValue == 6) ? true : relay6;
            relay7 = (integerValue == 7) ? true : relay7;
            relay8 = (integerValue == 8) ? true : relay8;
            rx_index = 0;
            memset(rx_buffer, 0, sizeof(rx_buffer));
        }
        HAL_UART_Receive_IT(&huart3, &rxData, 1);
    }
}
```

APPENDIX B | RELAY MODULE FIRMWARE LISTINGS

B.1 GPIO Initialization - MX_GPIO_Init()

```
static void MX_GPIO_Init(void) {
    GPIO_InitTypeDef GPIO_InitStructure = { 0 };

    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /* Set all outputs LOW before configuring mode */
    HAL_GPIO_WritePin(GPIOA,
        GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 |
        GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7 |
        GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_12 | GPIO_PIN_15,
        GPIO_PIN_RESET);

    HAL_GPIO_WritePin(GPIOB,
        GPIO_PIN_3 | GPIO_PIN_4 | GPIO_PIN_5 |
        GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8,
        GPIO_PIN_RESET);

    /* Configure GPIOA outputs */
    GPIO_InitStructure.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 |
        GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7 |
        GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_12 | GPIO_PIN_15;
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

    /* Configure GPIOB outputs */
    GPIO_InitStructure.Pin = GPIO_PIN_3 | GPIO_PIN_4 | GPIO_PIN_5 |
        GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStructure);
}
```

B.2 USART3 Receive Callback - Command Decoder

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART3) {
        if (rxData != 10) {
            rx_buffer[rx_index++] = rxData;
        } else if (strstr((char*) rx_buffer, "001")) {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 0); /* pump OFF */
        } else if (strstr((char*) rx_buffer, "101")) {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 1); /* pump ON */
        } else if (strstr((char*) rx_buffer, "002")) {
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, 0); /* fertilizer pump OFF */
        } else if (strstr((char*) rx_buffer, "102")) {
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, 1); /* fertilizer pump ON */
        } else if (strstr((char*) rx_buffer, "005")) {
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, 0); /* zone valve OFF */
        } else if (strstr((char*) rx_buffer, "105")) {
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, 1); /* zone valve ON */
        } else if (strstr((char*) rx_buffer, "006")) {
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, 0);
        } else if (strstr((char*) rx_buffer, "106")) {
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, 1);
        } else if (strstr((char*) rx_buffer, "007")) {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, 0);
        } else if (strstr((char*) rx_buffer, "107")) {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, 1);
        } else if (strstr((char*) rx_buffer, "008")) {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, 0);
        } else if (strstr((char*) rx_buffer, "108")) {
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, 1);
        } else if (strstr((char*) rx_buffer, "M0")) {
            if (position != 0) decrease = true;
        } else if (strstr((char*) rx_buffer, "M1")) {
            if (position != 90) increase = true;
        }
        rx_index = 0;
        memset(rx_buffer, 0, sizeof(rx_buffer));
        HAL_UART_Receive_IT(&huart3, &rxData, 1);
    }
}
```

B.3 Main Loop - Stepper Step Pulse Execution

```
while (1) {
    if (position != 0 && decrease) {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 1); /* direction A */
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 0);
        HAL_Delay(100);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 0);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 0);
        decrease = false;
        position--;
    }
    if (position != 90 && increase) {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 0);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 1); /* direction B */
        HAL_Delay(100);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 0);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 0);
        increase = false;
        position++;
    }
}
```

APPENDIX C | ESP32 ARCHITECTURE DESIGN SPECIFICATION

Status: Design complete - firmware implementation in progress.

Target: ESP32-WROOM-32

Framework: Arduino ESP32 Core (FreeRTOS)

Hardware connection to Sensor Module:

```
ESP32 GPIO16 (RX2) ← Sensor Module PB10 (USART3 TX)
ESP32 GPIO17 (TX2) → Sensor Module PB11 (USART3 RX)
Serial2.begin(9600, SERIAL_8N1, 16, 17);
```

Three-task FreeRTOS architecture:

```
void setup() {
    xTaskCreate(wifiManagerTask, "WiFiMgr", 4096, NULL, 1, NULL);
    xTaskCreate(mqttBridgeTask, "MQTTBridge", 8192, NULL, 2, NULL);
    xTaskCreate(stm32SerialTask, "STM32Ser", 4096, NULL, 2, NULL);
}
```

WiFi reconnection with exponential backoff:

```
void reconnectWiFi() {
    int delay_ms = 1000;
    while (WiFi.status() != WL_CONNECTED) {
        WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
        vTaskDelay(delay_ms / portTICK_PERIOD_MS);
        delay_ms = min(delay_ms * 2, 60000);
    }
    configTime(0, 0, "pool.ntp.org");
}
```

Sensor data JSON published to smartflow/sf2_001/sensors:

```
{
    "device_id": "sf2_001",
    "tds": <value>,
    "ph": <value>,
    "timestamp": "<ISO8601>"
}
```